Journal of Computational Physics 254 (2013) 8–26

Contents lists available at SciVerse ScienceDirect

Journal of Computational Physics

www.elsevier.com/locate/jcp

Robust untangling of curvilinear meshes

Thomas Toulorge [a,*], Christophe Geuzaine [b], Jean-François Remacle [a], Jonathan Lambrechts [a,c]

[a] Université catholique de Louvain, Institute of Mechanics, Materials and Civil Engineering (iMMC), Bâtiment Euler, Avenue Georges Lemaître 4, 1348 Louvain-la-Neuve, Belgium
[b] Université de Liège, Department of Electrical Engineering and Computer Science, Montefiore Institute B28, Grande Traverse 10, 4000 Liège, Belgium
[c] Fonds National de la Recherche Scientifique, rue d'Egmond, Bruxelles, Belgium

ARTICLE INFO

Article history:
Received 14 September 2012
Received in revised form 21 April 2013
Accepted 12 July 2013
Available online 29 July 2013

Keywords:
High-order methods
Finite elements
Mesh generation

ABSTRACT

This paper presents a technique that allows to untangle high-order/curvilinear meshes. The technique makes use of unconstrained optimization where element Jacobians are constrained to lie in a prescribed range through moving log-barriers. The untangling procedure starts from a possibly invalid curvilinear mesh and moves mesh vertices with the objective of producing elements that all have bounded Jacobians. Bounds on Jacobians are computed using the results of Johnen et al. (2012, 2013) [1,2]. The technique is applicable to any kind of polynomial element, for surface, volume, hybrid or boundary layer meshes. A series of examples demonstrate both the robustness and the efficiency of the technique. The final example, involving a time explicit computation, shows that it is possible to control the stable time step of the computation for curvilinear meshes through an alternative element deformation measure.

1. Introduction

There is a growing consensus in the computational mechanics community that state of the art solver technology requires, and will continue to require too extensive computational resources to provide the necessary resolution for a broad range of demanding applications, even at the rate that computational power increases. The requirement for high resolution naturally leads us to consider methods which have a higher order of grid convergence than the classical (formal) second order provided by most industrial grade codes. This indicates that higher-order discretization methods will replace at some point the current finite volume and finite element solvers, at least for part of their applications.

The development of high-order numerical technologies for engineering analysis has been underway for many years now. For example, discontinuous Galerkin methods (DGM) have been largely studied in the literature, initially in a theoretical context [3], and now from the application point of view [4]. In many contributions, it is shown that the accuracy of the method strongly depends on the accuracy of the geometrical discretization [5–7]. Consequently, it is necessary to address the problem of generating the high-order meshes that are needed to fully benefit from high-order methods.

Modern mesh generation procedures take as input CAD[1] models composed of *model entities*: vertices G_i^0, edges G_i^1, faces G_i^2 or regions G_i^3. Each model entity G_i^d has a geometry (or shape) [8,9] for which solid modelers usually provide

* Corresponding author.
E-mail address: thomas.toulorge@uclouvain.be (T. Toulorge).

[1] Computer Aided Design.

0021-9991/$ – see front matter
http://dx.doi.org/10.1016/j.jcp.2013.07.022

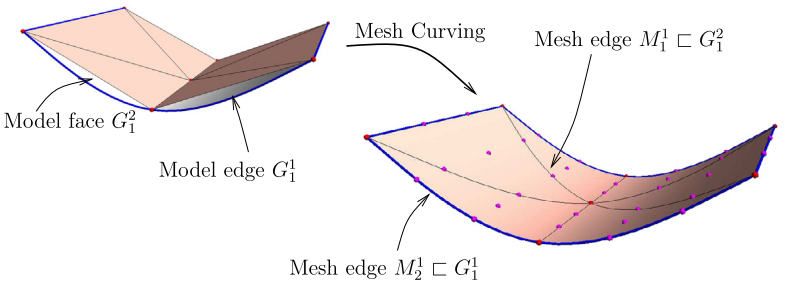

Fig. 1. Straight-sided mesh (left) and curvilinear (cubic) mesh (right).

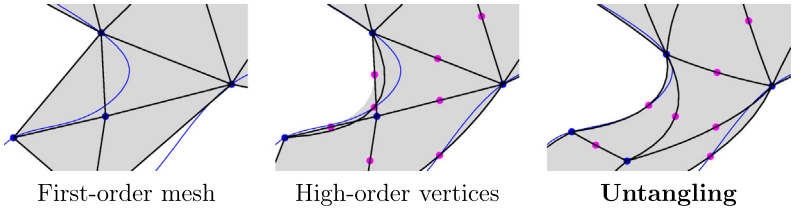

Fig. 2. Straight-sided mesh (left) basic curvilinear (quadratic) mesh with tangled elements (center) and untangled mesh (right).

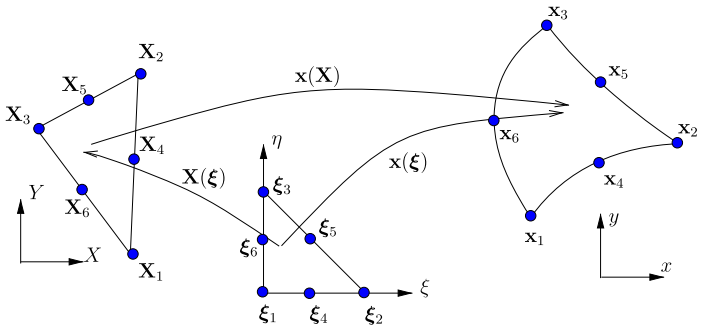

Fig. 3. Reference unit triangle in local coordinates $\boldsymbol{\xi} = (\xi, \eta)$ and the mappings $\mathbf{x}(\boldsymbol{\xi})$, $\mathbf{X}(\boldsymbol{\xi})$ and $\mathbf{X}(\mathbf{x})$.

a parametrization, that is, a mapping $\boldsymbol{\xi} \in R^d \mapsto \mathbf{x} \in R^3$. There are also four kind of mesh entities M_i^d that are said to be classified on model entities.[2] Each mesh entity is classified on the model entity of the smallest dimension that contains it.

The way of building a high-order mesh is to first generate a straight-sided mesh. Then, each mesh entity corresponding to a curved boundary of the domain is curved by adding high-order mesh points on the model entity on which it is classified (see Fig. 1). The position of the high-order nodes can be chosen in such a way that the geometrical error, i.e. the distance between the CAD model and the mesh, is minimized. In this paper however, the high-order nodes are simply defined by linear interpolation between the first-order nodes in the parametric space provided by the solid modeler for the corresponding model entity.

The naive curving procedure described above does not ensure that all the elements of the final curved mesh are valid. Fig. 2 gives an illustration of this important issue: some of the curved triangles are tangled after having been curved. It is important to note that this problem is not related to the accuracy of the geometrical discretization: in Fig. 2, the mesh would not be valid even if the curved edge was assigned the exact geometry (blue curve). Invalid elements may be detected by exploiting specific properties of the Jacobian of the transformations that map a fixed reference element onto each element in the physical domain, as shown in Fig. 3. In particular, a change of sign in the Jacobian over an element indicates that the corresponding map is not injective. In two recent papers [1,2], a general formulation has been developed for computing robust estimates of the geometrical validity of a curvilinear element. Provable bounds on element Jacobians can be computed for high-order triangles, quads, tetrahedra, hexahedra and prisms.

Fig. 2 indicates that, without refining the mesh, the only way of generating a valid high-order mesh is to curve not only mesh entities classified on curved model entities, but also those that are initially straight-sided inside the computational domain. It is thus necessary to somehow propagate the curvature inside the domain. Several smoothing schemes have been

[2] We use the symbol $\sqsubset$ for indicating that a mesh entity is classified on a model entity.

proposed in the literature to this effect: linear smoothing techniques such as Laplacian smoothing [10], Winslow smoothing [11] or linear elasticity with varying stiffness [10,12]. Even though such simple techniques may often lead to interesting results, there is no guarantee whatsoever that applying such a linear smoother will result in an untangled mesh. Nonlinear smoothing techniques have also been investigated, in particular using a nonlinear elasticity analogy [13], which results in a valid curvilinear mesh. Yet, computing a nonlinear mechanics problem including large deformations on a high-order and highly stretched mesh is numerically very complex. The computational cost may then be excessive, given that the meshing time is expected to be a fraction of the overall CPU time. Besides linear and nonlinear smoothing techniques, other authors [14–16] make use of mesh adaptation techniques by eliminating invalid elements by a combination of local mesh refinements, edge and face swaps, and node relocations.

In this paper, we propose a robust smoothing scheme that makes it possible to build a curvilinear mesh for which the validity of every element is guaranteed. This new untangling procedure relies on an optimization algorithm rather than a mechanical analogy or local mesh refinement: it specifically targets element Jacobians and modifies node locations in such a way that Jacobian values lie in a predefined range. This approach is loosely related to the optimization schemes proposed by several authors for untangling straight-sided meshes [17–20], but with a focus on arbitrarily high-order elements.

The paper is organized as follows. In Section 2, we briefly recall the results of paper [1,2] on Jacobian bounds, and explain how these bounds and their derivatives with respect to the motion of mesh vertices can be computed in practice. Section 3 is the kernel of the paper. An objective function that specifically targets invalid Jacobians is described in Section 3.1. Constraints on Jacobian positivity are imposed through log-barriers, allowing the use of unconstrained optimization procedures. The optimization strategy is described in Section 3.2. The optimization starts with an invalid mesh and the asymptote in the log-barrier is progressively moved into the valid region. Some examples of mesh untangling are presented in Section 4 where both the robustness and the efficiency of the new methodology are demonstrated. Some 3D examples are presented in Section 4.1 with timings and statistics. In Section 4.2, the example of a high-order boundary layer mesh is presented. A new term is added to the objective function with the aim of controlling the maximum Jacobian as well. Finally, Section 5 presents a simple idea for controlling the impact of mesh curvature on the explicit time step of computations.

2. Validity estimates of curvilinear meshes

2.1. Mesh validity and Jacobians

Let us introduce the following notations. We call n_e and n_v the number of elements and vertices of the mesh. Each element e of the mesh contains N_p vertices. Note that in this context, the word "vertex" denotes any node of the mesh, whether it is a geometrical vertex of a straight-sided element or a high-order node of a curved element.

We denote by $\mathbf{X}_i^e$ the position of the ith vertex of the element e in the straight-sided configuration and by $\mathbf{x}_i^e$ the location of the same vertex, yet in some deformed configuration.

The shape of an element e is defined geometrically through its vertices $\mathbf{x}_i^e$, $i = 1 \dots N_p$ that are nodes associated to a set of Lagrange shape functions $\mathcal{L}^{(p)}{}_i(\boldsymbol{\xi})$, $i = 1 \dots N_p$ at order p. This allows us to map a reference element to the real one:

$$\mathbf{x}(\boldsymbol{\xi}) = \sum_{i=1}^{N_p} \mathcal{L}^{(p)}{}_i(\boldsymbol{\xi}) \mathbf{x}_i^e. \tag{1}$$

Consider now the transformation $\mathbf{x}(\mathbf{X})$ that maps straight-sided elements onto curvilinear elements (see Fig. 3). The mapping $\mathbf{x}(\mathbf{X})$ should be bijective, i.e. it should admit an inverse. This implies that the determinant of the Jacobian $\det \mathbf{x}_{,\mathbf{X}}$ has to be non-zero, for every value of $\boldsymbol{\xi}$ in the reference element. In practice, this condition reduces to strict positivity for 3D elements, but an ambiguity remains for 2D elements, in which the Jacobian is defined with respect to a local normal. The orientation of planar 2D elements can be chosen in such a way that the direction of the constant normal ensures the positivity of the Jacobian. For curved surfaces in 3D however, the definition of the Jacobian is more complex, so that the condition of strict positivity that we retain in this paper may be overly restrictive.

It is possible to write this determinant in terms of the $\boldsymbol{\xi}$ coordinates as:

$$\det \mathbf{x}_{,\mathbf{X}} = \frac{\det \mathbf{x}_{,\boldsymbol{\xi}}}{\det \mathbf{X}_{,\boldsymbol{\xi}}} = \frac{J(\boldsymbol{\xi})}{J_0^e},$$

where J_0^e is the strictly positive and constant[3] Jacobian of the straight-sided element. The function $\mathbf{x}(\mathbf{X})$ is called the distortion mapping. Its determinant $\det \mathbf{x}_{,\mathbf{X}}$, that we call the *scaled Jacobian*, should be as close to 1 as possible in order not to degrade the quality of the straight-sided element e. Note however that the scaled Jacobian does not fully characterize the numerical properties of a high-order mesh, as shown in Section 5.

In [1,2] it is shown that it is possible to reliably detect invalid elements. In other words, it is possible to find reliable bounds to $J_{\min} = \min_{\boldsymbol{\xi}} J$ and to $J_{\max} = \max_{\boldsymbol{\xi}} J$ over the whole element. The Jacobian J is a polynomial in $\boldsymbol{\xi}$. It can then be

[3] Straight-sided element Jacobians are constant for simplicial elements only, i.e. triangles in 2D and tetrahedra in 3D. In the case of non-simplicial elements, J_0^e is taken as the value of the Jacobian at the centroid of the reference element.

interpolated exactly as a linear combination of Bézier polynomials $\mathcal{B}_i^{(q)}$ at a certain order $q \geqslant p$ over the element. Provable bounds for $J_{\min}$ and $J_{\max}$ are then computed using an interesting property of the Bézier polynomials, namely that their value is contained within the range of their coefficients. Assuming that J is a polynomial of order q in $\boldsymbol{\xi}$, we write

$$J(\boldsymbol{\xi}) = \sum_{i=1}^{N_q} \mathcal{B}_i^{(q)}(\boldsymbol{\xi}) B_i \tag{2}$$

and bounds can be computed as

$$\min_{\boldsymbol{\xi}} J(\boldsymbol{\xi}) \geqslant \min_i B_i \quad \text{and} \quad \max_{\boldsymbol{\xi}} J(\boldsymbol{\xi}) \leqslant \max_i B_i.$$

Although they are reliable, the bounds obtained in this manner may not be accurate: $\min_i B_i$ may be much lower than $\min_{\boldsymbol{\xi}} J(\boldsymbol{\xi})$ and $\max_i B_i$ may be much greater than $\max_{\boldsymbol{\xi}} J(\boldsymbol{\xi})$. The adaptive procedure described in [1,2] overcomes this problem, but it implies a higher computational cost, as well as additional complexity in the formulation. Therefore, we favor the direct evaluation provided by Eq. (2) in the present work, even though it yields an overly restrictive estimation of the Jacobian bounds.

The following section is dedicated to the practical computation of the coefficients B_i as well as their derivatives with respect to the vertex coordinates $\mathbf{x}_i^e$.

2.2. Computation of Bézier coefficients and their derivatives

The aim of our method is to untangle both surface and volume meshes. For that, we assume that a point $\mathbf{x}$ has always 3 coordinates $\mathbf{x} = \{x, y, z\}$. Local coordinates $\boldsymbol{\xi} = \{\xi, \eta, \zeta\}$ are also supposed to be three-dimensional. Yet, for surface meshes, we assume that vector

$$\mathbf{n} = \left\{ \frac{\partial x}{\partial \zeta}, \frac{\partial y}{\partial \zeta}, \frac{\partial z}{\partial \zeta} \right\}$$

is the constant unit normal vector to the straight-sided element. With that hypothesis, it is possible to compute the determinant of the Jacobian at every Lagrange node $\boldsymbol{\xi}_k = (\xi_k, \eta_k, \zeta_k)$ at order q:

$$J_k = J(\boldsymbol{\xi}_k) = \frac{\partial x}{\partial \xi}\frac{\partial y}{\partial \eta}\frac{\partial z}{\partial \zeta} + \frac{\partial z}{\partial \xi}\frac{\partial x}{\partial \eta}\frac{\partial y}{\partial \zeta} + \frac{\partial y}{\partial \xi}\frac{\partial z}{\partial \eta}\frac{\partial x}{\partial \zeta} - \frac{\partial z}{\partial \xi}\frac{\partial y}{\partial \eta}\frac{\partial x}{\partial \zeta} - \frac{\partial x}{\partial \xi}\frac{\partial z}{\partial \eta}\frac{\partial y}{\partial \zeta} - \frac{\partial y}{\partial \xi}\frac{\partial x}{\partial \eta}\frac{\partial z}{\partial \zeta}. \tag{3}$$

Considering that

$$x = \sum_{i=1}^{N_p} x_i^e \mathcal{L}^{(p)}{}_i(\boldsymbol{\xi}_k),$$

it is possible to compute the sensitivity of the Jacobian at point k with respect to the x coordinate of the vertex i:

$$\frac{\partial J_k}{\partial x_i^e} = \frac{\partial \mathcal{L}^{(p)}{}_i}{\partial \xi}\frac{\partial y}{\partial \eta}\frac{\partial z}{\partial \zeta} + \frac{\partial z}{\partial \xi}\frac{\partial \mathcal{L}^{(p)}{}_i}{\partial \eta}\frac{\partial y}{\partial \zeta} + \frac{\partial y}{\partial \xi}\frac{\partial z}{\partial \eta}\frac{\partial \mathcal{L}^{(p)}{}_i}{\partial \zeta} - \frac{\partial z}{\partial \xi}\frac{\partial y}{\partial \eta}\frac{\partial \mathcal{L}^{(p)}{}_i}{\partial \zeta} - \frac{\partial \mathcal{L}^{(p)}{}_i}{\partial \xi}\frac{\partial z}{\partial \eta}\frac{\partial y}{\partial \zeta} - \frac{\partial y}{\partial \xi}\frac{\partial \mathcal{L}^{(p)}{}_i}{\partial \eta}\frac{\partial z}{\partial \zeta}. \tag{4}$$

The same computation can be done for $\frac{\partial J_k}{\partial y_i^e}$ and $\frac{\partial J_k}{\partial z_i^e}$. In practice, the following matrix $\mathbf{J}$ of size $N_q \times (3N_p + 1)$ is computed for every element e:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial J_1}{\partial x_1^e} & \cdots & \frac{\partial J_1}{\partial x_{N_p}^e} & \frac{\partial J_1}{\partial y_1^e} & \cdots & \frac{\partial J_1}{\partial y_{N_p}^e} & \frac{\partial J_1}{\partial z_1^e} & \cdots & \frac{\partial J_1}{\partial z_{N_p}^e} & J_1 \\ \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots & \vdots \\ \frac{\partial J_{N_q}}{\partial x_1^e} & \cdots & \frac{\partial J_{N_q}}{\partial x_{N_p}^e} & \frac{\partial J_{N_q}}{\partial y_1^e} & \cdots & \frac{\partial J_{N_q}}{\partial y_{N_p}^e} & \frac{\partial J_{N_q}}{\partial z_1^e} & \cdots & \frac{\partial J_{N_q}}{\partial z_{N_p}^e} & J_{N_q} \end{bmatrix}.$$

Assuming that $T_{lk}^q = \mathcal{B}_l^{(q)}(\boldsymbol{\xi}_k)$ is the transformation matrix that enables to compute Bézier coefficients B_l using Lagrange coefficients J_k, the matrix

$$\mathbf{B} = \begin{bmatrix} \frac{\partial B_1}{\partial x_1^e} & \cdots & \frac{\partial B_1}{\partial x_{N_p}^e} & \frac{\partial B_1}{\partial y_1^e} & \cdots & \frac{\partial B_1}{\partial y_{N_p}^e} & \frac{\partial B_1}{\partial z_1^e} & \cdots & \frac{\partial B_1}{\partial z_{N_p}^e} & B_1 \\ \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots & \vdots \\ \frac{\partial B_{N_q}}{\partial x_1^e} & \cdots & \frac{\partial B_{N_q}}{\partial x_{N_p}^e} & \frac{\partial B_{N_q}}{\partial y_1^e} & \cdots & \frac{\partial B_{N_q}}{\partial y_{N_p}^e} & \frac{\partial B_{N_q}}{\partial z_1^e} & \cdots & \frac{\partial B_{N_q}}{\partial z_{N_p}^e} & B_{N_q} \end{bmatrix} \tag{5}$$

that contains both the Bézier coefficients B_l as well as their gradients with respect to the position of nodes of element e is calculated through a single matrix–matrix product: $B_{lj} = T_{lk}^q J_{kj}$.

It is then possible to use the B_i's and their gradients in a gradient-based optimization procedure. In what follows, an objective function that contains the coefficients B_i is constructed in order to control the quality of elements.

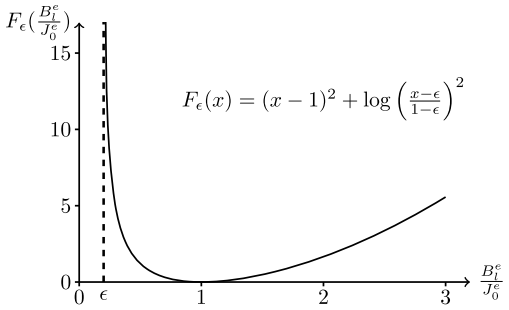

Fig. 4. Plot of the barrier function $F(J_l^e)$ for $\epsilon = 0.2$.

3. Curvilinear mesh untangling

3.1. An objective function

This section describes the objective function $f(\mathbf{x}_i^e)$ that is used to untangle invalid curved elements through an unconstrained optimization procedure. We design a function

$$f = \mathcal{E} + \mathcal{F}$$

that is composed of two parts $\mathcal{E}$ and $\mathcal{F}$.

Our assumption is that the method is provided with a straight-sided mesh of high quality. This mesh has potentially been defined to satisfy multiple criteria, such as a predetermined size field, or anisotropic adaptation. When curving such meshes, it is desirable to preserve as much as possible all these features, which implies keeping the nodes as close as possible to their initial positions in the straight-sided mesh.

To this end, we want to introduce some kind of *energy* $\mathcal{E}$ associated with the displacement of the nodes $\mathbf{x}_i^e - \mathbf{X}_i^e$, i.e. a positive quadratic form that is a measure of the distance between the straight-sided nodes $\mathbf{X}_i^e$ and their position $\mathbf{x}_i^e$ in the curved mesh:

$$\mathcal{E}(\mathbf{x}_i, \mathbf{K}) = \frac{1}{2}\sum_e \sum_{i=1}^{N_p}\sum_{j=1}^{N_p} \left(\mathbf{x}_i^e - \mathbf{X}_i^e\right)\mathbf{K}_{ij}^e\left(\mathbf{x}_j^e - \mathbf{X}_j^e\right) \geqslant 0 \tag{6}$$

where $\mathbf{K}$ is a symmetric positive matrix of size $3n_v \times 3n_v$ and $\mathbf{K}_{ij}^e$ is of size 3×3. In this paper, we choose $\mathbf{K}$ as the diagonal matrix with entries w_i^e/L^2, where w_i^e are user-defined weights used to set the balance between the two parts $\mathcal{E}$ and $\mathcal{F}$ of the objective function f, and L is a length scale representative of the problem. We choose L as the maximum distance, among all vertices of the initial tangled mesh, between a node and its counterpart in the straight-sided mesh. As for the non-dimensional weights w_i^e, we usually set $w_i^e = 10^5$ if the node i of element e lies on the boundary, and $w_i^e = 100$ otherwise. However, we observe in practice that the value of w_i^e has little influence on the convergence of the optimization method. The presence of $\mathcal{E}$ prevents the problem from being under-determined, and it orients the optimization procedure towards a solution that tends to preserve the straight-sided mesh, but the term $\mathcal{F}$ dominates for the most problematic (tangled) elements that drive the mesh deformation.

The second part $\mathcal{F}$ of the functional deals with Jacobian positivity. We use a log barrier [21] in order to avoid Jacobians that are too small, and a quadratic function to penalize Jacobians that are too high:

$$\mathcal{F}(\mathbf{x}_i, \epsilon) = \sum_{e=1}^{n_e}\sum_{l=1}^{N_q} F_l^e\left(\mathbf{x}_i^e, \epsilon\right)$$

with

$$F_l^e\left(\mathbf{x}_i^e, \epsilon\right) = \left[\log\left(\frac{B_l^e(\mathbf{x}_i^e) - \epsilon J_0^e}{J_0^e - \epsilon J_0^e}\right)\right]^2 + \left(\frac{B_l^e(\mathbf{x}_i^e)}{J_0^e} - 1\right)^2, \tag{7}$$

that is defined in such a way that $\mathcal{F}$ blows up when $B_l^e = \epsilon J_0^e$, but still vanishes whenever $B_l^e = J_0^e$. Barrier methods are among the most powerful class of algorithms available for attacking general nonlinear optimization problems. These techniques converge to at least a local minimum in most cases, regardless of the convexity characteristics of the objective function and constraints [22]. Fig. 4 shows a plot of the barrier function in Eq. (7) for $\epsilon = 0.2$.

The objective function f being smooth, it is possible to compute its gradient ∇f with respect to the positions of the element vertices $\mathbf{x}_i^e$. This gradient is used in a gradient-based optimization procedure.

Mesh vertices can be classified on mesh entities of various dimensions.

Some of the mesh vertices $M_i^0 \sqsubset G_j^1$ are classified on model edges. Such a vertex can only be moved along G_j^1, i.e. its position only depends on one curve parameter t. We have therefore

$$\frac{df}{dt} = \frac{\partial f}{\partial \mathbf{x}_i^e} \cdot \frac{d\mathbf{x}_i^e}{dt}$$

with $\frac{d\mathbf{x}_i^e}{dt}$ the tangent vector to the curve at point t, that is obtained from the CAD model.

Other vertices, that are classified on model faces $M_i^0 \sqsubset G_j^2$, can only be moved along the surface. Two parameters u and v are associated to those vertices. We have therefore

$$\frac{\partial f}{\partial u} = \frac{\partial f}{\partial \mathbf{x}_i^e} \cdot \frac{\partial \mathbf{x}_i^e}{\partial u} \quad \text{and} \quad \frac{\partial f}{\partial v} = \frac{\partial f}{\partial \mathbf{x}_i^e} \cdot \frac{\partial \mathbf{x}_i^e}{\partial v}$$

with $\frac{\partial \mathbf{x}_i^e}{\partial u}$ and $\frac{\partial \mathbf{x}_i^e}{\partial v}$ the two tangent vectors to the surface at point (u, v). Those can be computed using the CAD model.

Vertices that are classified on model regions have a complete freedom to move in every direction of the 3D space, in which case the physical coordinates $\{x, y, z\}$ are used. Finally, mesh vertices that are classified on model vertices have no freedom to move, and are excluded from the optimization problem.

3.2. Optimization strategy

The general optimization strategy is described in Algorithm 1. It involves a moving barrier for the objective function, preconditioning of the optimization problem and the selection of blobs of elements where the optimization procedure is to be applied locally.

In order to illustrate the robustness of our method and to put in evidence the influence of the different parameters, simple tests are performed on the annular geometry shown in Fig. 5. It is defined by two coaxial cylinders of radius R_{in} and R_{out} respectively, their length being set to R_{in}. Structured tetrahedral meshes are considered, with N_θ elements in the circumferential direction, N_r elements in the radial direction and one element in the axial direction. In the radial direction, the thickness of the layers of elements follows a geometrical progression of ratio α, in the fashion of boundary layer meshes used in CFD. In this configuration, both surface and volume nodes need to be moved in order to untangle the invalid elements resulting from the curving.

Algorithm 1: Optimization strategy.

```
Compute non-overlapping element blobs B_k, k = 1 ... N_B;
for k = 1 to N_B do
    repeat
        compute κ = min_e min_l B_l^e / J_0^e, e ∈ B_i, l ∈ [1, N_q];
        if κ < ε̄ then
            set ε = κ − 0.1|κ|;
            solve min_{x_i} f(x_i, K, ε) for all elements of blob B_k;
            recompute κ = min_e min_l B_l^e / J_0^e, e ∈ B_i, l ∈ [1, N_q];
    until κ ⩾ ε̄;
```

3.2.1. Optimization problem

The problem of untangling curvilinear meshes is defined as

$$\min_{\mathbf{x}_i} f(\mathbf{x}_i, \mathbf{K}, \epsilon), \quad i = 1, \ldots, n_v.$$

There is a variety of methods that can be used to solve unconstrained minimization problems. We have tested several alternatives: interior point methods implemented in the software package IPOPT [23], as well as L-BFGS [24] and conjugate gradients [25] algorithms provided by ALGLIB [25]. In the end, the use of conjugate gradients seemed to be the best choice in terms of computational efficiency.

The most important part of the optimization strategy is to define the right sequence of optimization problems. In order for the variables to remain in the domain of definition of the barrier function $\mathcal{F}$, the value of the barrier ϵ must be lower than the worst scaled Jacobian of all elements in one given blob. In particular, ϵ has to be negative for an initially tangled mesh. Therefore, we compute a sequence of optimization problems with "moving barriers": ϵ is increased between each optimization problem, until the desired barrier value $\bar{\epsilon}$ is reached. This procedure is illustrated in Fig. 6.

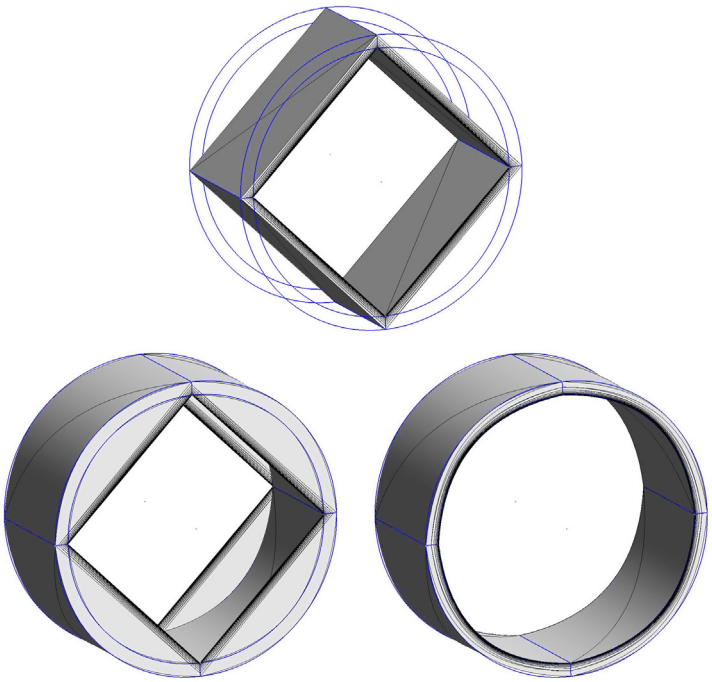

Fig. 5. Annular geometry with $R_{in} = 1$ and $R_{out} = 1.1$: straight-sided mesh (top), primary second-order mesh (left) and untangled mesh (right) with $N_\theta = 4$ elements in the circumferential direction and $N_r = 20$ layers of elements along the radius (geometric progression of ratio $\alpha = 1.3$).

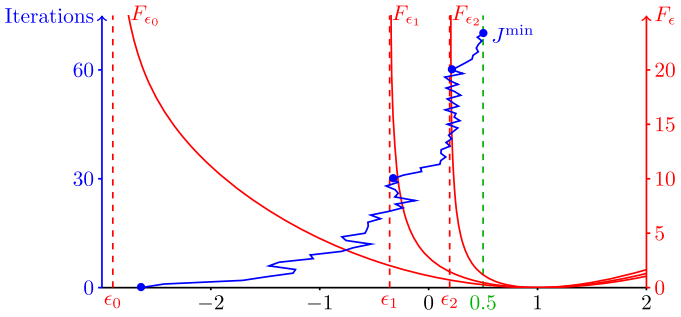

Fig. 6. Optimization process: three successive series of (maximum) 30 conjugate gradient iterations, with their respective log-barriers.

Table 1
Parameters of the meshes for the tests of the untangling procedure on the annular geometry.

α	γ	ρ	$J^e_{\min}/J^e_0$
1.1	$9.8 \cdot 10^{-4}$	$1.1 \cdot 10^{-3}$	−847
1.2	$3.2 \cdot 10^{-4}$	$3.7 \cdot 10^{-4}$	−2564
1.3	$1.0 \cdot 10^{-4}$	$1.2 \cdot 10^{-4}$	−8019

We apply the untangling procedure to the annular geometry described above with $R_{in} = 1$ and $R_{out} = 1.1$. The circumference is covered by only $N_\theta = 4$ elements, so that the straight-sided mesh resembles a shell of square cross-section, as illustrated in Fig. 5. $N_r = 19$ layers of elements are used along the radius, and the geometric progression ratio α for the layer thickness is comprised between 1.1 and 1.3. Thus, each mesh contains 456 tetrahedral elements. In order to characterize the quality of the straight-sided mesh before curving, we consider the ratio γ of the inscribed radius to the circumscribed radius, as well as the ratio ρ of the minimum edge length to the maximum edge length, for the worst element. The minimum scaled Jacobian $J^e_{\min}/J^e_0$ of the primary quadratic mesh is also calculated. These parameters are listed for each test in Table 1.

The untangling procedure is run for each mesh with a target scaled Jacobian $\bar{\epsilon}$ of 0.1, 0.5 and 0.9. The maximum number of conjugate gradient iterations for the before moving the barrier is set to 1500. Table 2 summarizes the results in terms of number of iterations, CPU time, and minimum scaled Jacobian $J^e_{\min}/J^e_0$ in the final mesh.

Table 2
Results for the tests of the untangling procedure on the annular geometry.

α	$\bar{\epsilon}$	No. iter.	CPU time [s]	$J^e_{\min}/J^e_0$
1.1	0.1	300	2.7	0.17
	0.5	325	3.0	0.81
	0.9	$1500+1$	12.3	0.93
1.2	0.1	1144	9.9	0.18
	0.5	1350	11.9	0.67
	0.9	$2\times1500+2$	26.2	0.79
1.3	0.1	$4\times1500+231$	57.8	0.13
	0.5	$4\times1500+1413$	68.1	0.67
	0.9	$7\times1500+1164$	106.1	0.60

Table 3
Results for the preconditioning tests of the untangling procedure on the annular geometry.

Geometric scale	Preconditioning		No preconditioning	
	No. iter.	CPU time [s]	No. iter.	CPU time [s]
10^{-6}	331	3.3	358	3.3
10^{-4}	331	3.4	329	3.3
10^{-2}	348	3.4	337	3.4
1	325	3.0	385	3.7
10^{2}	342	3.2	–	–
10^{4}	346	3.2	–	–
10^{6}	338	3.3	–	–

The untangling procedure manages to provide valid meshes for this challenging problem. It is clear that the convergence is slower when the mesh contains elements that are more stretched (i.e. with larger α) for a given scaled Jacobian target $\bar{\epsilon}$. For each mesh, the computational effort required to reach $\bar{\epsilon}=0.1$ and $\bar{\epsilon}=0.5$ is similar. However, the highly constrained nature of the problem, that is due to the limited thickness of the annulus, makes it much more costly for the procedure to reach $\bar{\epsilon}=0.9$.

Table 2 may look surprising: the final value of the minimum scaled Jacobian $J^e_{\min}/J^e_0$ may differ significantly from the target $\bar{\epsilon}$ that is reached at the end of the optimization procedure: it is higher for the easiest cases (i.e. low α and low $\bar{\epsilon}$), but lower for the difficult cases ($\bar{\epsilon}=0.9$). This is because the Jacobians are scaled by a constant value J^e_0 computed on the initial mesh in the optimization procedure. Yet, corner vertices of elements may be changed during the optimization process, leading to possible changes in values of straight-sided Jacobians J^e_0. Thus, the final value of $J^e_{\min}/J^e_0$ can differ from the threshold, but the positivity of the Jacobian is not threatened as long as the optimization procedure is successful.

3.2.2. Preconditioning

It is necessary to apply preconditioning to the optimization problem, because the scale of parametric or physical coordinates of different mesh vertices can differ by orders of magnitude, depending on the model entity on which they are classified. We found that a simple diagonal preconditioner, based on the norm of the tangent vectors $\frac{d\mathbf{x}^e_i}{dt}$ for vertices classified on model edges (respectively $\frac{\partial\mathbf{x}^e_i}{\partial u}$ and $\frac{\partial\mathbf{x}^e_i}{\partial v}$ for vertices classified on model faces), and unity for vertices classified on model regions, yields fast and robust convergence.

In order to verify the effect of the preconditioning, we apply the untangling procedure to the problem described above with $\alpha=0.1$, with annular geometries of different scales. The computational effort needed to reach the target scaled Jacobian of $\bar{\epsilon}=0.5$ is measured, and the results with and without preconditioning are listed in Table 3. The method without preconditioning works well for geometries of small scale, but fails to converge in a reasonable amount of time for scale greater than 1. In contrast, the method with preconditioning converges at the same rate regardless of the geometric scale.

3.2.3. Blobs

Applying the untangling procedure described above to the whole mesh results in a robust methodology. In practical cases however, it often represents a waste of computational resources. Indeed, only the vertices located near the curved geometry need to be moved in order to untangle the mesh, while the vertices located far away remain mostly unchanged. For the purpose of improving the computational efficiency of the method, the optimization procedure is thus applied only locally.

After identifying all invalid elements in the mesh, the first step consists in building a blob of N layers around each of them. These blobs are usually appropriate in isotropic meshes, but not in boundary layer-type meshes, where elements have high aspect ratio. There, the boundary deformation responsible for the mesh tangling is typically large compared to the normal size of the tangled element, but small compared to its tangential size (see Fig. 7). In order to untangle such a mesh, several layers must be curved in the direction normal to the boundary, whereas modifying the elements that are adjacent in the tangential direction is useless. Therefore, a geometrical criterion is taken into account in the construction of the blobs,

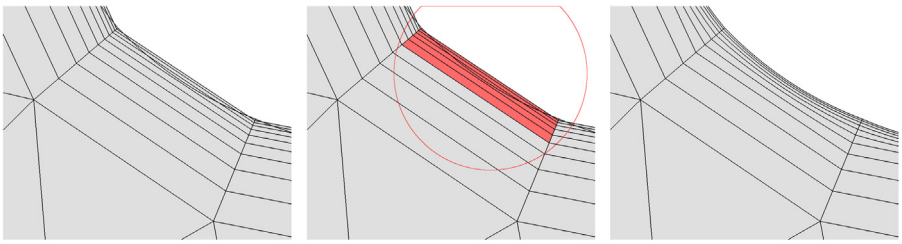

Fig. 7. Detail of a boundary-layer mesh on a curved geometry: tangled second-order mesh (left), blob definition with a circle representing the geometrical criterion (center), and untangled mesh (right).

as illustrated in Fig. 7: among the elements contained in the N layers surrounding the invalid element, only those located within a certain distance are retained. This distance is defined by multiplying the distance between the straight-sided and high-order boundaries by a user-defined factor. The boundaries of the blob are fixed not to take any risk of invalidating elements outside the blob.

The optimal values for the number of layers N and the distance factor are case-dependent. In practical applications with boundary-layer meshes, we found that $N = 6$ layers of quadrangles (or $N = 12$ layers of triangles) and a distance factor of 12 are usually sufficient to reach a typical scaled Jacobian of $\bar{\epsilon} = 0.3$. In order to ensure the robustness of the method, it is possible to set up an iterative process in which the size of the blob is progressively increased if the untangling procedure fails to reach the scaled Jacobian target $\bar{\epsilon}$. This did not prove necessary in any of the examples presented in this paper.

4. Examples of mesh untangling

4.1. 3D Mechanical parts

Fig. 8 shows images of the mesh of a mechanical part before and after the untangling process. The mesh is composed of 5605 quadratic tetrahedra of which 215 have a $J^e_{\min}/J^e_0 < 0.3$. The untangling process was performed in 13 blobs of elements obtained by considering $N = 2$ layers around invalid elements.

In the example of Fig. 8, the total CPU time for untangling that mesh is 18 seconds on a standard laptop. A global optimization procedure involving one single blob of 5,605 elements takes 60 seconds to converge.

Another example is presented in Fig. 9. The same parameters as in the first example is used. The total time needed to untangle the 3 blobs of elements is now 23 seconds.

4.2. High-lift airfoil

We consider the high-lift airfoil shown in Fig. 12. Two straight-sided meshes $\mathcal{M}_1$ and $\mathcal{M}_2$, featuring boundary layer-type stretching around every surface of the airfoil, are generated. $\mathcal{M}_1$ is made out of 9814 triangles, while $\mathcal{M}_2$ contains 3178 triangles and 3318 quads. Both meshes are converted to order 2 to 5 and optimized.

The objective function described in Section 3.1 is designed to untangle invalid elements that are created by the conversion of straight-sided meshes to higher order, which happens mainly near convex parts of the geometry. However, we note that the conversion can also generate elements of lower quality close to concave parts, as seen in Fig. 10. These elements are not strictly invalid, because their Jacobian is clearly positive everywhere, but they do not maintain the boundary layer-type size progression imposed in the straight-sided mesh. In order to remedy this inconvenience, we supplement the procedure described in Section 3 with a second optimization pass that controls the maximum Jacobian in the mesh. In the first pass, the objective function f introduced in Section 3.1 is used in the "moving barrier" procedure mentioned in Section 3.2 to reach the desired minimum Jacobian $\bar{\epsilon}_{\min}$. In the second pass, we switch to a different objective function f_1:

$$f_1(\mathbf{x}_i, \mathbf{K}, \bar{\epsilon}_{\min}, \bar{\epsilon}_{\max}) = \mathcal{E}(\mathbf{x}_i, \mathbf{K}) + \mathcal{F}(\mathbf{x}_i, \bar{\epsilon}_{\min}) + \mathcal{F}(\mathbf{x}_i, \bar{\epsilon}_{\max})$$

where the term $\mathcal{F}(\mathbf{x}_i, \bar{\epsilon}_{\min})$ maintains the minimum Jacobian constraint at the value $\bar{\epsilon}_{\min}$, while the term $\mathcal{F}(\mathbf{x}_i, \bar{\epsilon}_{\max})$ (plotted in Fig. 11) is handled as a moving barrier that is decreased to reach the desired maximum Jacobian $\bar{\epsilon}_{\max}$. As a result, the elements located near the concave parts of the geometry also curve to adapt the high-order boundary, as shown in Fig. 10.

We optimize meshes $\mathcal{M}_1$ and $\mathcal{M}_2$ with $\bar{\epsilon}_{\min} = 0.4$ and $\bar{\epsilon}_{\max} = 1.6$ at order $p = 2 \ldots 5$. Given the element aspect ratio and grid stretching in the boundary layer zone for this test case, we obtain good results with blobs of $N = 6$ layers of quads (or $N = 12$ layers of triangles) and a distance factor of 12 for the creation of blobs. Information about the results is provided in Table 4. The number of operations needed to compute f and its derivatives depends on the size of matrix $\mathbf{B}$ of Eq. (5), i.e. it is a function of $N_p \times N_q$. In Table 4, we report N_p and N_q for the different mesh orders p. It must

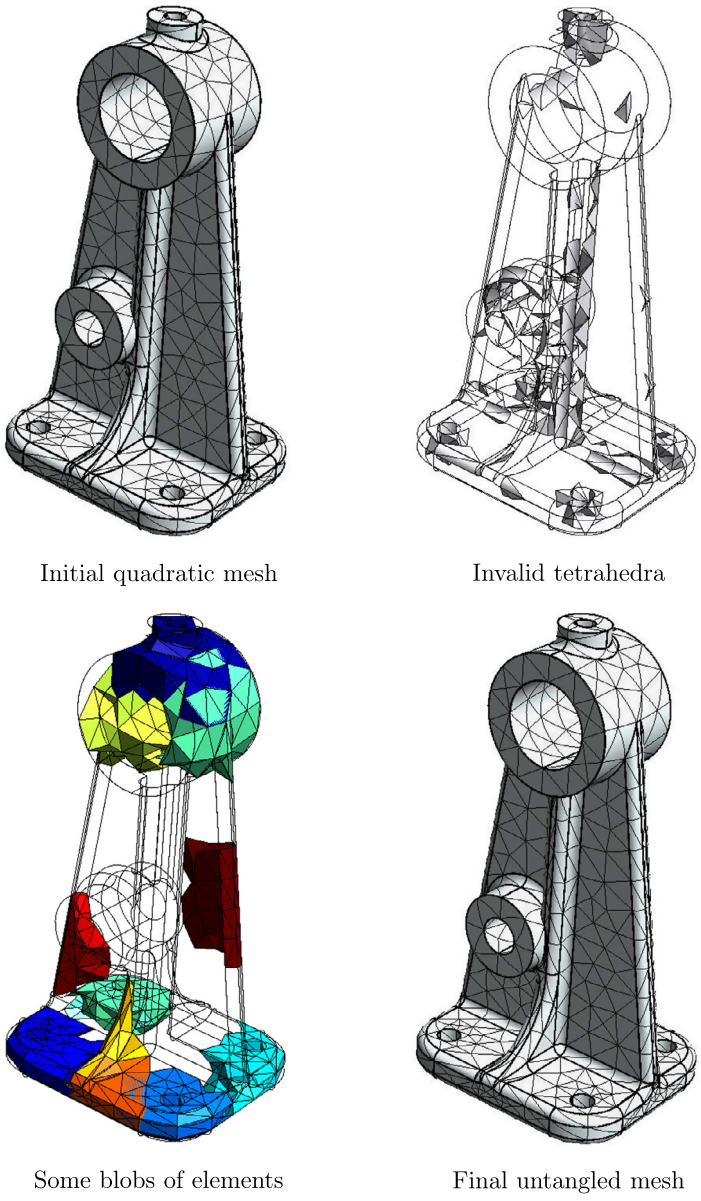

Fig. 8. Mesh of a mechanical part that is composed of 5605 quadratic tetrahedra. Figures show a surface view of the initial quadratic mesh, a volume view of the 215 invalid tetrahedra, a view of 10 among 13 blobs of elements used in the optimization process and the final untangled mesh.

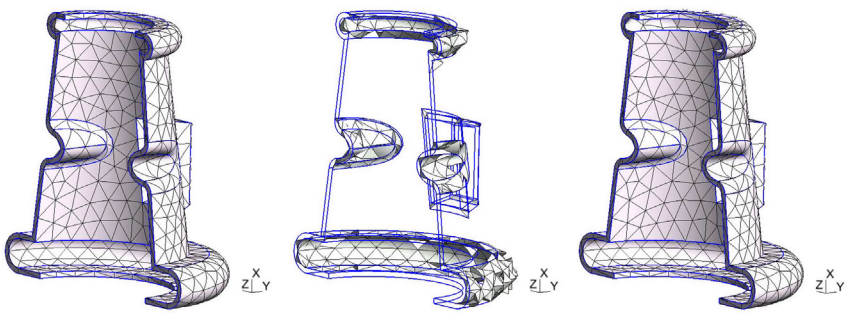

Fig. 9. Mesh of a mechanical part that is composed of 2988 quadratic tetrahedra. Figures show a surface view of the initial quadratic mesh, a volume view of the 131 invalid tetrahedra and the final untangled mesh.

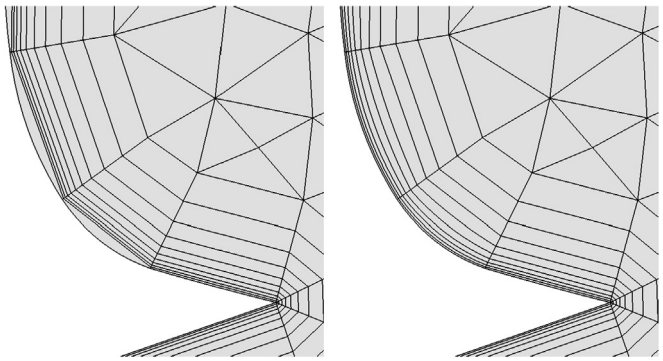

Fig. 10. Detail of the mesh on the suction side of the slat in the high-lift airfoil case: primary second-order mesh (left), optimized mesh (right).

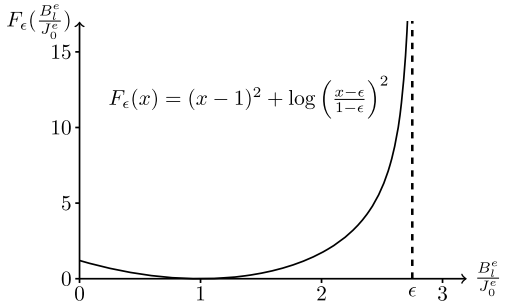

Fig. 11. Plot of the barrier function $F(J_l^e)$ for $\epsilon = 2.75$.

Table 4
Statistics for the high-lift airfoil test case.

Mesh	p	q	N_p	N_q	$N_{invalid}$	n_v	t (s)	$J^e_{\min}/J^e_0$
$\mathcal{M}_1$	2	2	6	6	51	20,751	3.0	0.40
$\mathcal{M}_1$	3	4	10	15	56	45,452	8.2	0.40
$\mathcal{M}_1$	4	6	15	28	64	79,967	20.0	0.40
$\mathcal{M}_1$	5	8	21	45	74	124,296	65.2	0.41
$\mathcal{M}_2$	2	3	9	16	32	20,751	1.4	0.55
$\mathcal{M}_2$	3	5	16	36	34	45,452	7.0	0.50
$\mathcal{M}_2$	4	7	25	64	31	79,967	36.3	0.46
$\mathcal{M}_2$	5	9	36	100	31	124,296	330.7	0.44

be noted that, for mesh $\mathcal{M}_2$, all elements in the boundary layer are quads, so that N_p and N_q correspond to quads. The number of Bézier points N_q for triangles corresponds to order $q = 2(p-1)$ while the Jacobians of quads are of order $q = 2p-1$. It is therefore not surprising that the computation time for untangling quartic meshes is about 10 times larger than to untangle a quadratic mesh. Quintic meshes ($p = 5$) however are disappointingly expensive to compute for quads. This problem may be essentially related to the bad conditioning of the optimization problem. In our implementation, we use Lagrange equidistant shape functions, which are known to be highly oscillatory for high orders. The use of polynomial basis with a better Lebesgue constant such as those of Ref. [26] may improve the results. Nevertheless, our procedure remains quite efficient for high-order meshes of orders that are required for engineering analysis, i.e. $p = 2$ and $p = 3$ [27,28]. As mentioned in Section 3.2, the final value of the minimum scaled Jacobian $J^e_{\min}/J^e_0$ generally differs from the target value $\bar{\epsilon}_{\min} = 0.4$, without any consequence for the mesh validity.

Finally, Fig. 12 shows a comparison between the linear elastic analogy with uniform material properties and the present procedure for the mesh $\mathcal{M}_1$ at $p = 2$: using the elastic analogy, only a few layers of elements are affected by the process, leaving there a large number of invalid elements. It is indeed possible to understand mathematically why approaches that rely on elliptic boundary value problems for smoothing may fail in such a configuration. Consider Fig. 3. The change in the configuration of a the triangle results in a displacement

$$\mathbf{U}(\mathbf{x}) = \mathbf{X}(\mathbf{x}) - \mathbf{x}$$

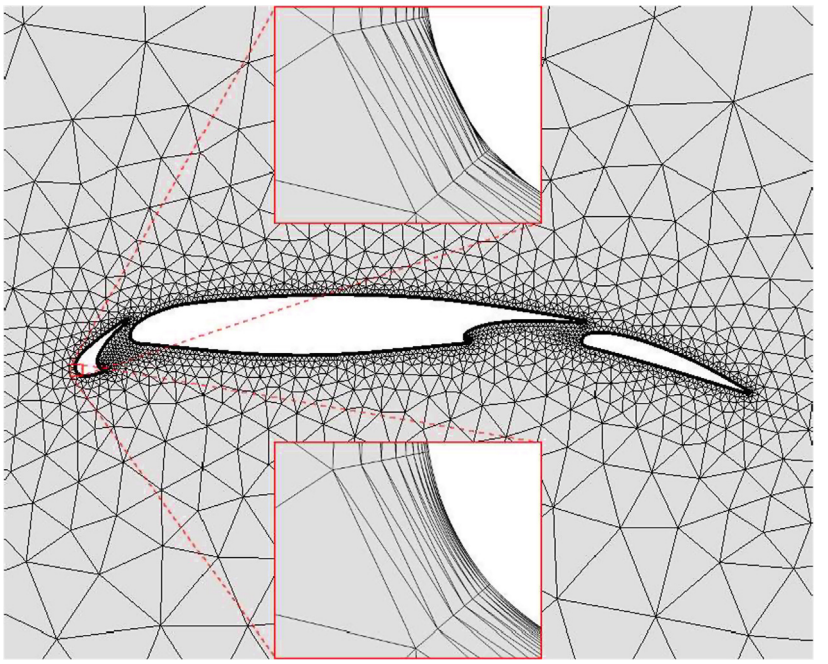

Fig. 12. Quadratic triangular mesh for the high-lift airfoil case. Zoom at the leading edge of the slat: comparison between the mesh obtained by elastic analogy (upper frame) and by the present optimization procedure (lower frame).

that allow to compute an elastic energy $E(\mathbf{U})$ that can be minimized with respect to $\mathbf{U}$.[4] The value of $\mathbf{U}$ is imposed on every vertex that is classified on the boundary of the domain:

- If a vertex is not a high-order vertex, then $\mathbf{U} = \mathbf{0}$,
- If a vertex is a high-order vertex, then $\mathbf{U}$ is imposed as the displacement that enables to move the vertex on the real geometry.

This kind of highly oscillatory boundary condition cannot propagate very far into the domain due to the elliptic nature of the elastic operator (see [29]): the "wave length" of the boundary condition being of the order of the mesh size, the information on the boundary cannot propagate into the domain further than a distance about the mesh size. This illustrates why the simple elastic analogy approach may fail in presence of highly stretched boundary layer elements [13], whereas the optimization procedure manages to untangle the mesh by propagating the high-order boundary deformation through many neighboring layers of elements.

4.3. Nozzle

In this section, we consider the 3D geometry of a nozzle, where both the interior and the exterior of the nozzle are meshed, as illustrated in Fig. 13. A linear hybrid mesh, featuring boundary layers on the inner and outer walls of the nozzle, is generated by extrusion of a 2D mesh. It is composed of 125,340 prisms and 52,240 hexahedra.

The conversion of the straight-sided mesh to second order yields 3780 invalid elements, located on the outer surface of the geometry. The untangling procedure is applied with a target minimum scaled Jacobian of $\bar{\epsilon} = 0.1$, while the maximum scaled Jacobian is disregarded. Two blobs are created from the 4160 elements that have an initial scaled Jacobian lower than $\bar{\epsilon}$.

[4] Linear elastic models use the tensor of small deformations

$$\boldsymbol{\epsilon} = \frac{1}{2}\left(\frac{d\mathbf{U}}{d\mathbf{x}}^T + \frac{d\mathbf{U}}{d\mathbf{x}}\right) = \frac{1}{2}\left(\frac{d\mathbf{X}}{d\mathbf{x}}^T + \frac{d\mathbf{X}}{d\mathbf{x}}\right) - \mathbf{I}$$

and use the Hooke tensor $\mathbf{C}$ for computing elastic stresses $\boldsymbol{\sigma} = \mathbf{C}\boldsymbol{\epsilon}$. The elastic energy is defined as

$$E = \frac{1}{2}\int_{\Omega} \boldsymbol{\sigma} : \boldsymbol{\epsilon}\, d\mathbf{X}.$$

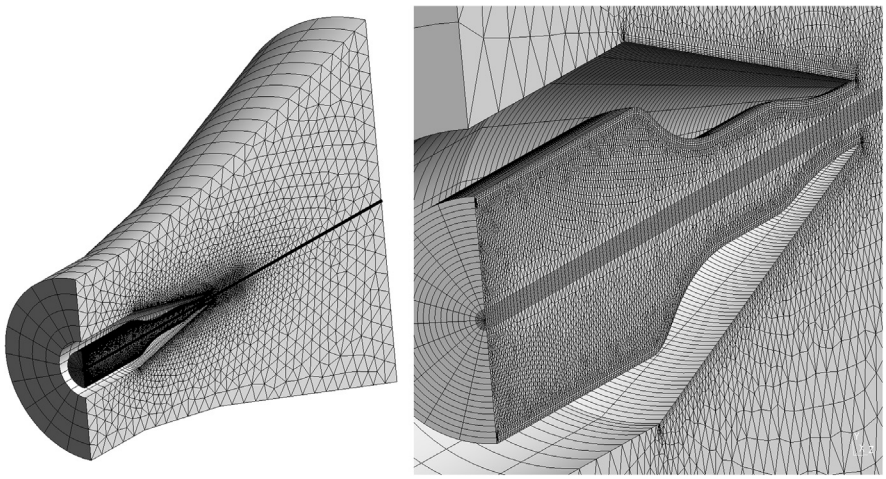

Fig. 13. Mesh for the nozzle case: general view (left), zoom on the nozzle geometry (right).

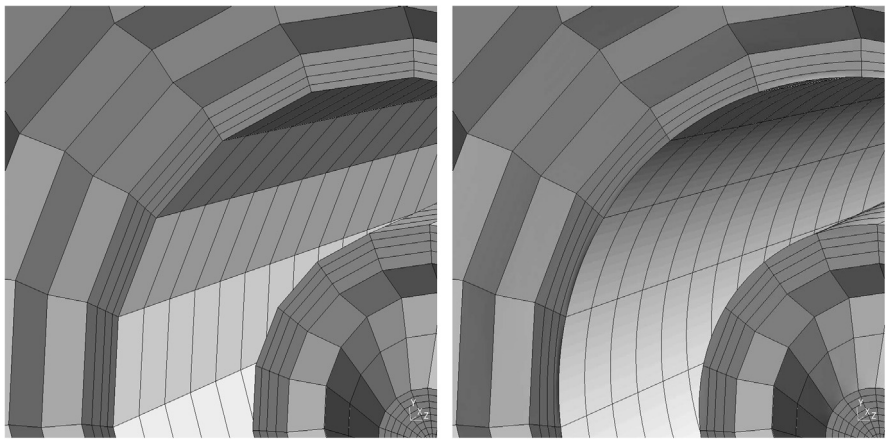

Fig. 14. Detail of the linear (left) and final quadratic mesh (right) for the nozzle case. Note that a limitation in the visualization of volume elements is responsible for the irregular aspect of the curved mesh.

Because of technical limitations related to the parametrization of the geometry by the solid modeler, boundary nodes are fixed. The optimization procedure succeeds to untangle the mesh in about 23 min, resulting in a minimum scaled Jacobian of $J^e_{\min}/J^e_0 = 0.13$. A detail of the mesh is shown in Fig. 14.

5. Mesh curvature impact on the explicit time step: a simple idea

5.1. Another measure of deformation

It is shown in Ref. [30] that the conditioning of the operators deriving from high-order spatial discretization methods can be affected by the curvature of the elements. When using implicit time integration in simulations, this effect has a negative impact on the convergence of iterative linear algebra solvers. With explicit time stepping schemes, it reduces the maximum time step allowed by the stability restrictions, as explained more thoroughly in Section 5.2.

The optimization procedure described in Section 3 is very efficient at producing meshes with scaled Jacobians that are all positive and close to 1. In mathematical terms, the procedure tends to generate mappings $\mathbf{x}(\mathbf{X})$ that are one-to-one and close to be *equiareal* (in 2D). As mentioned in Section 3.1, this is due to the term $\mathcal{F}$ that dominates the objective function f for invalid elements. In comparison, the part $\mathcal{E}$ is small, so that the vertices can be moved enough to fix the tangled elements. In other words, the optimization procedure tries to untangle high-order elements by recovering the area in 2D (or the volume in 3D) that they have in the initial straight-sided mesh.

In some cases, however, the approximate conservation of the area or volume may not prevent an untangled element to end up with a wildly different shape from the original straight-sided one, as illustrated in Fig. 15. This may affect the numerical behavior of the simulations using such meshes. For instance, consider the three quadratic triangles shown in Fig. 16, that have a scaled Jacobian strictly constant and equal to one. It is clear that those triangles have very different

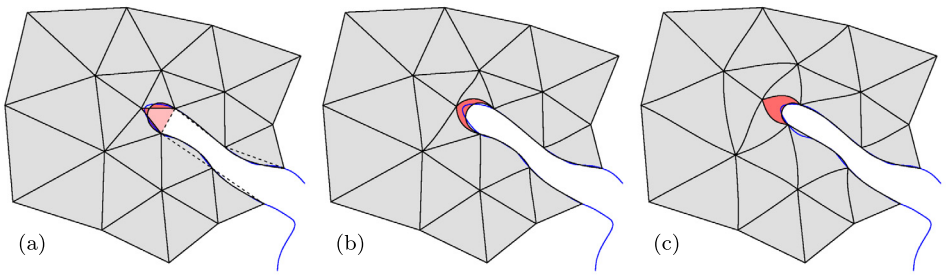

Fig. 15. Typical optimization of a two-layer patch around an invalid element of the Great Barrier Reef third-order coarse mesh: (a) initial patch and original linear mesh (in dashed lines), (b) first optimization based on the Jacobian determinant and (c) second optimization based on the minimum eigenvalue of the metric matrix $\mathcal{M}$. The blue line is the coastline given by the CAD model. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

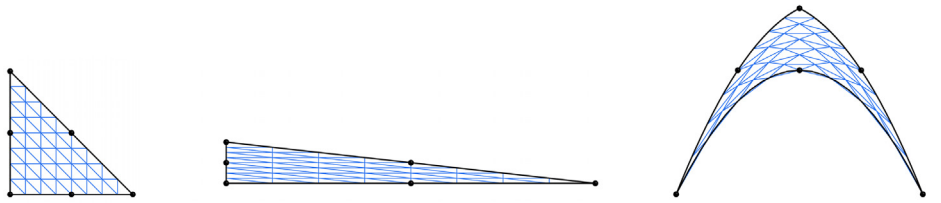

Fig. 16. Three second-order triangles with the same area and the same constant Jacobian determinant but obviously leading to different stable time step.

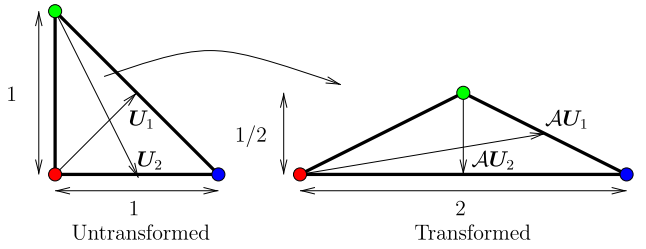

Fig. 17. Two triangles that have the same area. We have here $\frac{\|\mathcal{A}\mathbf{U}_1\|}{\|\mathbf{U}_1\|} = \frac{\sqrt{37}}{2\sqrt{2}} \simeq 2.15$ which is close to $\sqrt{\lambda^{\max}(\mathcal{M})} = \frac{\sqrt{21+\sqrt{377}}}{2\sqrt{2}} \simeq 2.247$ and $\frac{\|\mathcal{A}\mathbf{U}_2\|}{\|\mathbf{U}_2\|} = \frac{1}{\sqrt{5}} \simeq 0.447$ which is close to $\sqrt{\lambda^{\min}(\mathcal{M})} = \frac{\sqrt{21-\sqrt{377}}}{2\sqrt{2}} \simeq 0.445$.

interpolation properties in a finite element computation. Prescribing the Jacobian of the elements is thus required to ensure the mesh validity, but it is clearly not sufficient for controlling the mesh quality. The mesh optimization procedure should also be able to control element lengths, in order to avoid problematic cases like the one shown in Fig. 15.

Consider the two triangles of Fig. 17. Let $\mathcal{A} = \mathbf{x}_{,\mathbf{X}}$ be the constant Jacobian matrix of the transformation between the two triangles. The untransformed triangle and the transformed triangle have the same surface ($\det \mathcal{A} = 1$). The first fundamental form or metric tensor of the mapping

$$\mathcal{M} = \mathcal{A}\mathcal{A}^* \tag{8}$$

allows to compute variations of lengths. Assume $\mathbf{U}$ to be a vector that defines a direction in the undeformed space (see Fig. 17). Vector $\mathbf{U}$ transforms in $\mathcal{A}\mathbf{U}$ as it is depicted on the figure. The ratio between the lengths of the two vectors is written as

$$l = \frac{\|\mathcal{A}\mathbf{U}\|}{\|\mathbf{U}\|} = \sqrt{\frac{\mathbf{U}^*\mathcal{M}\mathbf{U}}{\mathbf{U}^*\mathbf{U}}}.$$

Then, we have

$$l^{\min} = \min_{\|\mathbf{U}\|>0} \frac{\|\mathcal{A}\mathbf{U}\|}{\|\mathbf{U}\|} = \sqrt{\lambda^{\min}(\mathcal{M})}$$

where $\lambda^{\min}(\mathcal{M})$ is the smallest eigenvalue of $\mathcal{M}$.

Thus, considering the eigenvalues of the metric tensor in addition to the Jacobian gives a better characterization of the mesh quality. This problem is well posed in the sense that the only mapping that result in a metric tensor that has three equal eigenvalues (e.g. the isometry) does not deform the element. In this perspective, we extend our approach not only to

obtain positive Jacobians but also to control the minimum eigenvalue of $\mathcal{M}$. We thus look locally at how element lengths are transformed and try to control the minimum value $\lambda^{\min}$ on all the element.

In two dimensions, $\mathcal{M}$ is expressed as:

$$\mathcal{M} = \begin{pmatrix} \|x_{,\mathbf{x}}\|^2 & x_{,\mathbf{x}} \cdot y_{,\mathbf{x}} \\ x_{,\mathbf{x}} \cdot y_{,\mathbf{x}} & \|y_{,\mathbf{x}}\|^2 \end{pmatrix}.$$

And its smallest eigenvalue, $\lambda^{\min}$ can be obtained easily:

$$\lambda^{\min}(\mathcal{M}) = \|x_{,\mathbf{x}}\|^2 + \|y_{,\mathbf{x}}\|^2 - \sqrt{\left(\|x_{,\mathbf{x}}\|^2 - \|y_{,\mathbf{x}}\|^2\right)^2 + 4(x_{,\mathbf{x}} \cdot y_{,\mathbf{x}})^2}.$$

For each element blob, we introduce a second stage to the optimization process described in Section 3. After the optimization of the Jacobian in the first stage, the same procedure is repeated with a modified objective function:

$$f_2(\mathbf{x}_i, \mathbf{K}, \bar{\epsilon}, \zeta) = \mathcal{E}(\mathbf{x}_i, \mathbf{K}) + \mathcal{F}(\mathbf{x}_i, \bar{\epsilon}) + \mathcal{G}(\mathbf{x}_i, \zeta).$$

The term $\mathcal{F}(\mathbf{x}_i, \bar{\epsilon})$ maintains the minimum Jacobian constraint $\bar{\epsilon}$, while the additional term $\mathcal{G}$ controls the minimum eigenvalue of the metric tensor:

$$\mathcal{G}(\mathbf{x}_i, \zeta) = \sum_{e=1}^{n_e} \sum_{l=1}^{N_q} G_l^e\left(\mathbf{x}_i^e, \zeta\right),$$

$$G_l^e\left(\mathbf{x}_i^e, \zeta\right) = \left[\log\left(\frac{\lambda^{\min}(\mathbf{x}_i^e) - \zeta}{1 - \zeta}\right)\right]^2 + \left(\lambda^{\min}\left(\mathbf{x}_i^e\right) - 1\right)^2.$$

Unfortunately, due to the square root, $\lambda^{\min}$ is not polynomial. Therefore, it cannot be directly expressed as a sum of Bézier polynomials to guarantee that it is above the objective value everywhere on the element. In practice, the value of $\lambda^{\min}$ is sampled at the same points as the Jacobian and these additional constraints, combined with those on the Jacobian, are enough to preserve the quality of the elements.

Assuming the usual expansion of Eq. (1) for $\mathbf{x}$, the derivative of $\lambda^{\min}$ with respect to nodal positions can be computed as

$$\lambda^{\min}(\mathcal{M})_{,x_i^e} = 2\left(x_{,\mathbf{x}} - \frac{(\|x_{,\mathbf{x}}\|^2 - \|y_{,\mathbf{x}}\|^2)x_{,\mathbf{x}} + 2(x_{,\mathbf{x}} \cdot y_{,\mathbf{x}})y_{,\mathbf{x}}}{\sqrt{(\|x_{,\mathbf{x}}\|^2 - \|y_{,\mathbf{x}}\|^2)^2 + 4(x_{,\mathbf{x}} \cdot y_{,\mathbf{x}})^2}}\right) \cdot \mathcal{L}^{(p)}{}_{i,\mathbf{x}},$$

$$\lambda^{\min}(\mathcal{M})_{,y_i^e} = 2\left(y_{,\mathbf{x}} - \frac{(\|y_{,\mathbf{x}}\|^2 - \|x_{,\mathbf{x}}\|^2)y_{,\mathbf{x}} + 2(x_{,\mathbf{x}} \cdot y_{,\mathbf{x}})x_{,\mathbf{x}}}{\sqrt{(\|x_{,\mathbf{x}}\|^2 - \|y_{,\mathbf{x}}\|^2)^2 + 4(x_{,\mathbf{x}} \cdot y_{,\mathbf{x}})^2}}\right) \cdot \mathcal{L}^{(p)}{}_{i,\mathbf{x}}.$$

Fig. 15 shows how optimization on $\lambda^{\min}$ affects the resulting meshes. The left figure shows the initial invalid mesh. Algorithm 1 is then applied, resulting in the mesh of the central figure. This mesh, while perfectly valid, has elements that are dramatically shrunk in some directions. Finally, optimization on $\lambda^{\min}$ is applied, resulting in the mesh of the right figure. This mesh has element shapes which are very close to those of the straight-sided mesh.

5.2. Impact of the deformation with explicit time stepping

It is mentioned in Section 5.1 that the deformation of high-order elements may affect the numerical methods used in simulations through the mapping $\mathbf{x}(\boldsymbol{\xi})$. Hereafter, we explain how these effects can be characterized by the deformation measure introduced in Section 5.1, in the particular case of numerical methods using explicit time stepping schemes to resolve unsteady phenomena.

With explicit time stepping schemes, the curvature of the elements reduces the maximum time step allowed by the stability restrictions through the Jacobian matrix $\mathbf{x}_{,\boldsymbol{\xi}}$ of the mapping $\mathbf{x}(\boldsymbol{\xi})$, as shown in Ref. [30]. This is illustrated in Fig. 18, where the evolution of the maximum time step is plotted while progressively deforming a grid, for a simple advection problem solved with a DG spatial discretization and an explicit Runge–Kutta time integrator. Thus, setting a scaled Jacobian barrier $\bar{\epsilon}$ closer to 1 tends to mitigate the adverse effects of high-order mesh curvature.

However, in the simple example of Fig. 18, the elements are deformed in a constant direction. The general results of Ref. [30], as well as our experience with high-order computations, suggest that there is no simple relation between the conditioning of the semi-discrete operator and the Jacobian $\det \mathbf{x}_{,\boldsymbol{\xi}}$.

Consider a simple scalar advection equation, as a model for more general hyperbolic systems: find $u(\mathbf{x}, t)$ solution of

$$\frac{\partial u}{\partial t} + \mathbf{V} \cdot \nabla_{\mathbf{x}} u = 0. \tag{9}$$

The explicit integration in time of the ordinary differential equation resulting from the discretization of the spatial operator $\mathbf{V} \cdot \nabla_{\mathbf{x}}$ is subject to conditional stability. Linear stability conditions are usually more restrictive than nonlinear stability conditions, so the stability results obtained for the conservation law (9) can be applied to nonlinear partial differential equations

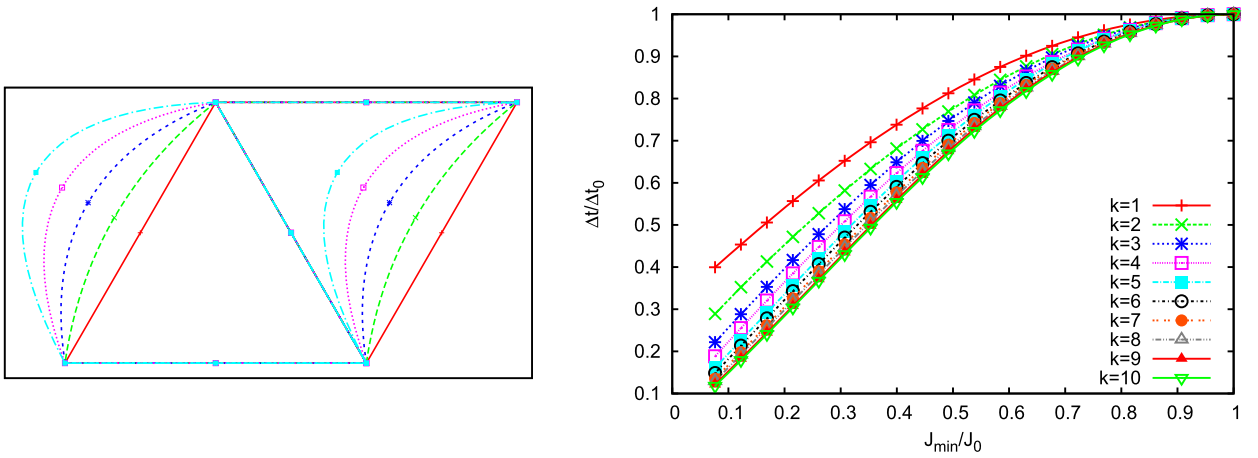

Fig. 18. Impact of the progressive deformation of a structured grid for a scalar advection problem, solved with the classic explicit 4-stage Runge–Kutta time integrator and a DG spatial discretization of order k ranging from 1 to 10: deformation of the pattern of triangular elements of which the structured grid is made up (left), evolution of the time step Δt (scaled by the time step Δt_0 for straight-sided elements) as a function of the scaled Jacobian $\frac{J_{\min}}{J_0}$ (right). More details can be found in Ref. [30].

(PDEs). These conditions can further be generalized to systems of hyperbolic PDEs by considering that u corresponds to a characteristic variable of the system and $\mathbf{V}$ corresponds to the associated characteristic velocity. The stability of the system is then driven by the characteristic leading to the most restrictive stability condition. However, the systems of multidimensional PDEs of practical interest often have an infinite set of characteristics forming a Monge cones instead of degenerating into lines. Thus, we are interested in a stability condition that takes all possible directions of $\mathbf{V}$ into consideration, i.e. a condition on the norm $\|\mathbf{V}\|$ of $\mathbf{V}$.

This so-called Courant–Friedrichs–Levy (CFL) condition can be defined as:

$$\|\mathbf{V}\|\Delta t \leqslant C\Delta x \tag{10}$$

where Δt is the stable time step, Δx is a length that represents the element size, and C a constant that depends on the numerical method [31]. This condition should be fulfilled everywhere on the element. In the case of straight-sided triangular elements, Δx is usually chosen as the inner radius of the triangle.

How does this size definition extend to curvilinear meshes? In order to find a stability condition of the advection problem on the curved element, we transform it into a problem on a straight-sided element for which we can use the stability condition (10). The conservation law (9) is written in a system of coordinates, $\mathbf{x}(\mathbf{X})$, which can be seen as the transformation of another one, $\mathbf{X}$ corresponding to a straight-sided element. The Jacobian of this transformation is $\mathcal{A} = \mathbf{x}_{,\mathbf{X}}$. In the system of coordinates $\mathbf{X}$, the conservation law (9) reads:

$$\frac{\partial u}{\partial t} + \underbrace{\mathbf{V}\cdot\mathcal{A}^{-1}}_{\mathbf{V}'}\cdot\nabla_{\mathbf{X}}u = 0. \tag{11}$$

The problems (9) and (11) being equivalent, the stability condition (10) can be written in the system of coordinates $\mathbf{X}$ as:

$$\|\mathbf{V}'\|\Delta t = \|\mathbf{V}\cdot\mathcal{A}^{-1}\|\Delta t < C\Delta X$$

where ΔX is the size of the straight-sided element. Let $l^{\min}$ be the smallest singular value of $\mathcal{A}$ so that $\frac{1}{l^{\min}}$ is the largest singular value of $\mathcal{A}^{-1}$. We have:

$$\|\mathbf{V}\mathcal{A}^{-1}\| < \frac{\|\mathbf{V}\|}{l^{\min}}.$$

The condition:

$$\|\mathbf{V}\|\,\Delta t < C\frac{\Delta X}{l^{\min}} \tag{12}$$

is then sufficient to satisfy the stability condition (12). Condition (10) can be seen as the usual CFL condition (10) with an element size $\Delta x = \Delta X/l^{\min}$ depending on the smallest singular value $l^{\min}$ of the mapping $\mathcal{A}$, i.e. the smallest eigenvalue of the metric tensor $\mathcal{M} = \mathcal{A}\mathcal{A}^T$, as defined in Section 5.1.

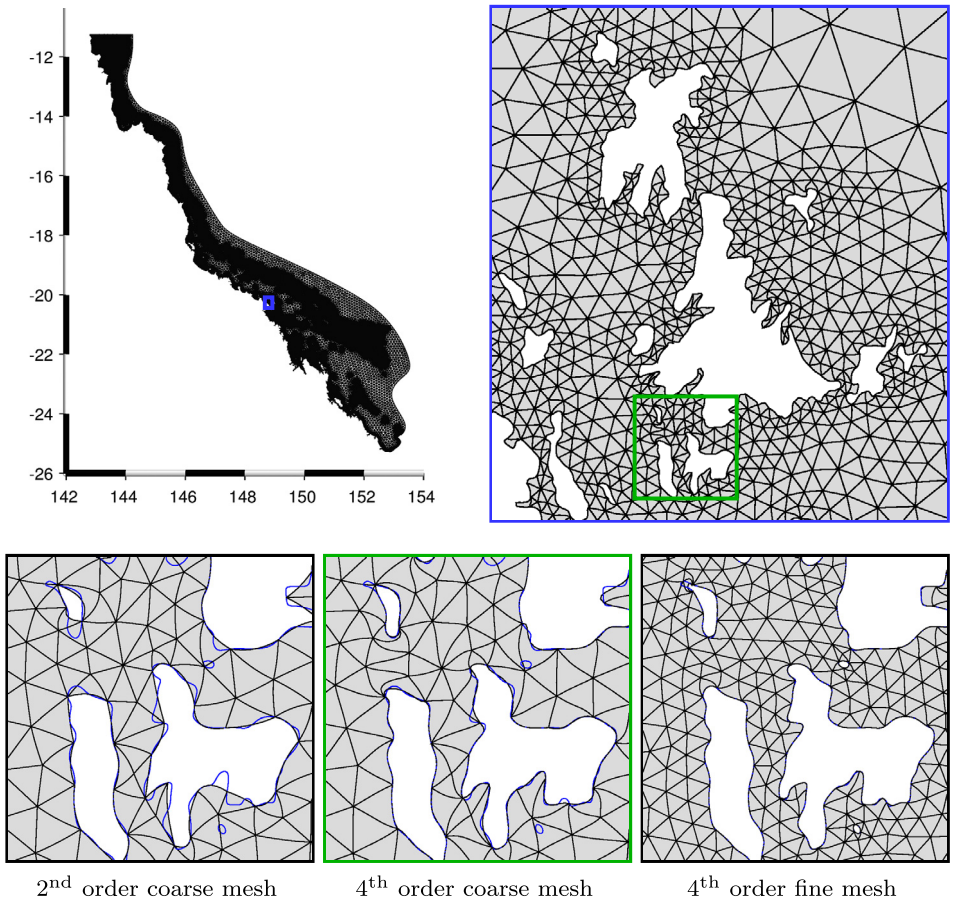

Fig. 19. Top: fourth-order coarse curvilinear mesh (88k triangles) of the Great Barrier Reef, Australia and detail on the Whitsunday islands. Below: closer zoom on Hamilton Island for different meshes.

5.3. Application on a large-scale example

In order to show the interest of the deformation measure introduced in Sections 5.1 and 5.2 for practical applications, we apply the mesh optimization procedure to a large-scale oceanic simulation.

The patch of Fig. 15 is a very small part of a larger mesh of the Great Barrier Reef (GBR), in Australia. The GBR is composed of more than 2500 reefs and islands. This complex topography generates circulation patterns on a wide range of scales and makes the generation of a curved mesh of this domain particularly challenging. Second-, third- and fourth-order versions of two linear meshes of this region are considered. The first mesh is composed of about 330,000 triangles and the second one, even coarser, is composed of about 88,000 triangles. Those meshes are presented in Fig. 19. In order to test our method on a difficult case, we use in both meshes a coarse resolution compared to the small features of the underlying geometrical model, so that highly curved elements are generated near the boundaries.

On each mesh, we determine the maximum stable time step to solve the shallow water equations using a discontinuous Galerkin method of the same polynomial order as the mesh with a fourth-order explicit Runge–Kutta temporal scheme. The details of the parameterization and the boundary conditions can be found in [32]. For each mesh, an estimation of the maximum time step is progressively refined through a simple bisection method that determines the stability by running the simulation for a few iterations and checking the global norm of the solution.

Table 5 shows the different time steps obtained on linear meshes, on curvilinear meshes optimized with respect to the scaled Jacobians only and on meshes further optimized with respect to $\lambda^{\min}$. By chance, the Jacobian-based optimization is enough to recover the same time step on the second-order mesh as on the linear mesh with a requirement of $J^e_{\min}/J^e_0 = 0.5$. However, this is not the case for the other meshes, even when a high scaled Jacobian is imposed. On the contrary, on all meshes optimized first with respect to the scaled Jacobians and then with respect to $\lambda^{\min}$ (using an objective value of 0.5 in both optimizations), the maximum stable time step is very close to the maximum stable time step of the same finite element method on the linear mesh. The CPU time taken by the optimization process ranges from 5 seconds for the second-order coarse mesh to about 10 minutes for the fourth-order fine mesh.

Table 5
Maximum stable time step to solve the shallow water with discontinuous Galerkin method of polynomial order 2 to 4 and an explicit fourth-order Runge–Kutta, on linear meshes, curvilinear meshes optimized for the Jacobians only and curvilinear meshes further optimized for the metric. The optimization process stops when all the Jacobians or the eigenvalues of the metric are greater than 0.1 or 0.5.

	p	$J^e_{\min}/J^e_0$		$\lambda^{\min}$		Linear
		0.1	0.5	0.1	0.5	
Coarse	2	1.7	3.0	2.5	3.1	3.0
	3	0.8	0.9	2.0	2.0	1.9
	4	0.5	0.5	1.2	1.2	1.2
Fine	2	0.8	1.0	1.4	3.1	3.0
	3	0.4	0.5	1.4	2.0	2.0
	4	0.4	0.5	0.9	1.3	1.4

6. Conclusions

The ability of generating high-order/curvilinear mesh is a prerequisite to the generalization of high-order numerical methods in engineering analysis. This paper offers a framework that allows to generate such meshes in a robust and efficient way.

The methodology consists in solving a sequence of optimization problems in which the deformation of high-order mesh elements, compared to their linear counterpart, is minimized. The deformation measure is based on the Jacobian of the transformation that maps a high-order element onto the straight-sided one, which enables to strictly controls the mesh validity. Optionally, the minimum eigenvalue of the corresponding metric can be used in a second step, with positive impact on simulations involving explicit time stepping. Constraints on the maximum deformation are imposed through moving barriers, in order to guarantee the quality of the high-order mesh generated. The restriction of the optimization process to blobs of elements surrounding a tangled element, instead of the whole mesh, make the methods computationally efficient.

We have tested the new untangling procedure on various other examples than those presented in this paper, and our conclusions can be summarized as follows:

- The use of moving log-barriers for imposing Jacobian constraints is clearly a better solution than using constrained optimization, both in terms of computational cost and of robustness.
- The energy part (6) of the objective function is not a critical parameter in the untangling process, its only purpose being most of the time to ensure a unique solution to the optimization process.
- It is of paramount importance to allow mesh vertices to move on the model entities they are classified on. This is especially true in 3D where both surface and volume meshes should be untangled at once.
- The untangling of quadratic meshes using the new technique is very fast, robust and not sensitive to any parameter that we have introduced. We consider that issue to be well addressed.

The method works with 2D meshes composed of quadrangles and hexahedra, as well as 3D meshes containing tetrahedra, hexahedra and prisms. In the case of pyramidal elements, robust Jacobian bounds cannot be directly obtained from Bézier coefficients as described in Section 2, but the technique can be extended to take into account the rational nature of the shape functions. We are currently working on this topic. In any case, pyramidal elements are normally not found in the close vicinity of the geometry, so their curvature usually remains limited, and sampling the Jacobian may yield sufficiently reliable bounds.

All the material that is presented in this paper is readily available in Gmsh, the open source mesh generator [9]. We are confident that users will help us to improve further Gmsh's high-order mesh generation capabilities. Nevertheless, some direct extensions of this work are already planned.

First, the objective function should somehow take into account the geometrical accuracy of the high-order mesh representations. High-order vertices should allow to reduce some norm of the distance between the mesh and the CAD model.

Then, the influence of element shapes on the numerical solutions should be investigated. In this paper, we have shown that it is possible to compute a length quantity representing the size of a curvilinear element and use it to compute the CFL condition, but the high-order nature of meshes can also influence the approximation properties of numerical schemes [33]. We should thus investigate the influence of mesh curving on the quality of numerical solutions, especially in computational fluid dynamics.

Acknowledgements

This research project was funded in part by the Walloon Region under WIST 3 grant 1017074 "DOMHEX" (Dominant Hexahedral Mesh Generation) and in part by the European Union under the FP7 grant "IDIHOM" (Industrialisation of High-Order Methods – A Top-Down Approach). The authors are grateful to Koen Hillewaert (CENAERO) for providing the nozzle geometry and the mesh settings used in Section 4.3.

References

[1] A. Johnen, J.-F. Remacle, C. Geuzaine, Geometrical validity of curvilinear finite elements, in: W.R. Quadros (Ed.), Proceedings of the 20th International Meshing Roundtable, Springer, Berlin, Heidelberg, 2012, pp. 255–271.

[2] A. Johnen, J.-F. Remacle, C. Geuzaine, Geometrical validity of curvilinear finite elements, Journal of Computational Physics 233 (2013) 359–372, http://dx.doi.org/10.1016/j.jcp.2012.08.051.

[3] B. Cockburn, G. Karniadakis, C.-W. Shu (Eds.), Discontinuous Galerkin Methods, Lecture Notes in Computational Science and Engineering, vol. 11, Springer, Berlin, 2000.

[4] N. Kroll, H. Bieler, H. Deconinck, V. Couaillier, H. Van Der Ven, K. Sorensen, ADIGMA – A European Initiative on the Development of Adaptive Higher-Order Variational Methods for Aerospace Applications: Results of a Collaborative Research Project Funded by the European Union, 2006–2009, Notes on Numerical Fluid Mechanics and Multidisciplinary Design, Springer, 2010.

[5] F. Bassi, S. Rebay, High-order accurate discontinuous finite element solution of the 2D Euler equations, Journal of Computational Physics 138 (2) (1997) 251–285, http://dx.doi.org/10.1006/jcph.1997.5454.

[6] P.-E. Bernard, J.-F. Remacle, V. Legat, Boundary discretization for high-order discontinuous Galerkin computations of tidal flows around shallow water islands, International Journal for Numerical Methods in Fluids 59 (5) (2009) 535–557, http://dx.doi.org/10.1002/fld.1831.

[7] T. Toulorge, W. Desmet, Curved boundary treatments for the discontinuous Galerkin method applied to aeroacoustic propagation, AIAA Journal 48 (2) (2010) 479–489, http://dx.doi.org/10.2514/1.45353.

[8] J.-F. Remacle, M.S. Shephard, An algorithm oriented mesh database, International Journal for Numerical Methods in Engineering 58 (2) (2003) 349–374, http://dx.doi.org/10.1002/nme.774.

[9] C. Geuzaine, J.-F. Remacle, Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities, International Journal for Numerical Methods in Engineering 79 (11) (2009) 1309–1331.

[10] P.L. George, H. Borouchaki, Construction de maillages de degré 2. Partie 3: Tétraèdre P2, Tech. rep., INRIA, France, 2011.

[11] P.M. Knupp, Winslow smoothing on two-dimensional unstructured meshes, Engineering with Computers 15 (1999) 263–268, http://dx.doi.org/10.1007/s003660050021.

[12] Z.Q. Xie, R. Sevilla, O. Hassan, K. Morgan, The generation of arbitrary order curved meshes for 3D finite element analysis, Computational Mechanics 51 (3) (2013) 361–374, http://dx.doi.org/10.1007/s00466-012-0736-4.

[13] P.-O. Persson, J. Peraire, Curved mesh generation and mesh refinement using lagrangian solid mechanics, in: Proceedings of the 47th AIAA Aerospace Sciences Meeting and Exhibit, Orlando (FL), USA, 5–9 January 2009, 2009.

[14] X. Li, M. Shephard, M. Beall, Accounting for curved domains in mesh adaptation, International Journal for Numerical Methods in Engineering 58 (2) (2003) 247–276.

[15] X. Luo, M. Shephard, R. O'Bara, R. Nastasia, M. Beall, Automatic p-version mesh generation for curved domains, Engineering with Computers 20 (3) (2004) 273–285.

[16] M. Shephard, J. Flaherty, K. Jansen, X. Li, X. Luo, N. Chevaugeon, J. Remacle, M. Beall, R. O'Bara, Adaptive mesh generation for curved domains, Applied Numerical Mathematics 52 (2) (2005) 251–271.

[17] L.A. Freitag, P. Plassmann, Local optimization-based simplicial mesh untangling and improvement, International Journal for Numerical Methods in Engineering 49 (1) (2000) 109–125.

[18] E.J. López, N.M. Nigro, M.A. Storti, J.A. Toth, A minimal element distortion strategy for computational mesh dynamics, International Journal for Numerical Methods in Engineering 69 (2007) 1898–1929.

[19] E.J. López, N.M. Nigro, M.A. Storti, Simultaneous untangling and smoothing of moving grids, International Journal for Numerical Methods in Engineering 76 (7) (2008) 994–1019.

[20] L. Diachin, P. Knupp, T. Munson, S. Shontz, A comparison of two optimization methods for mesh quality improvement, Engineering with Computers 22 (2) (2006) 61–74.

[21] L. Freitag, P. Knupp, T. Munson, S. Shontz, A comparison of optimization software for mesh shape-quality improvement problems, Tech. rep., Argonne National Lab., IL, USA, 2002.

[22] A. Fiacco, G. McCormick, Nonlinear Programming: Sequential Unconstrained Minimization Techniques, vol. 4, Society for Industrial Mathematics, 1990.

[23] A. Waechter, C. Laird, F. Margot, Y. Kawajir, Introduction to IPOPT: A tutorial for downloading, installing, and using IPOPT, 2009.

[24] D. Liu, J. Nocedal, On the limited memory BFGS method for large scale optimization, Mathematical Programming 45 (1) (1989) 503–528.

[25] R. Fletcher, C. Reeves, Function minimization by conjugate gradients, Computer Journal 7 (2) (1964) 149–154.

[26] J. Hesthaven, T. Warburton, Nodal Discontinuous Galerkin Methods: Algorithms, Analysis, and Applications, Texts in Applied Mathematics, vol. 54, Springer-Verlag New York Inc., 2008.

[27] P. Vos, S. Sherwin, R. Kirby, From h to p efficiently: Implementing finite and spectral hp element methods to achieve optimal performance for low- and high-order discretisations, Journal of Computational Physics 229 (13) (2010) 5161–5181.

[28] P. Bernard, J. Remacle, R. Comblen, V. Legat, K. Hillewaert, High-order discontinuous Galerkin schemes on general 2D manifolds applied to the shallow water equations, Journal of Computational Physics 228 (17) (2009) 6514–6535.

[29] J.-F. Remacle, C. Geuzaine, G. Compère, E. Marchandise, High-quality surface remeshing using harmonic maps, International Journal for Numerical Methods in Engineering 83 (4) (2010) 403–425.

[30] T. Toulorge, W. Desmet, Spectral properties of discontinuous Galerkin space operators on curved meshes, in: J.S. Hesthaven, E.M. Rønquist, T.J. Barth, M. Griebel, D.E. Keyes, R.M. Nieminen, D. Roose, T. Schlick (Eds.), Spectral and High Order Methods for Partial Differential Equations, in: Lecture Notes in Computational Science and Engineering, vol. 76, Springer, Berlin, Heidelberg, 2011, pp. 495–502.

[31] B. Seny, J. Lambrechts, R. Comblen, V. Legat, J.-F. Remacle, Multirate time stepping for accelerating explicit discontinuous Galerkin computations with application to geophysical flows, International Journal for Numerical Methods in Fluids 71 (1) (2013) 41–64, http://dx.doi.org/10.1002/fld.3646.

[32] J. Lambrechts, E. Hanert, E. Deleersnijder, P.-E. Bernard, V. Legat, J.-F. Remacle, E. Wolanski, A multiscale model of the whole Great Barrier Reef hydrodynamics, Estuarine, Coastal and Shelf Science 79 (2008) 143–151, http://dx.doi.org/10.1016/j.ecss.2008.03.016.

[33] L. Botti, Influence of reference-to-physical frame mappings on approximation properties of discontinuous piecewise polynomial spaces, Journal of Scientific Computing 52 (3) (2012) 675–703, http://dx.doi.org/10.1007/s10915-011-9566-3.